

Resolução de Problemas por Meio de Busca

Agentes solucionadores de problemas

Prof. Talles Medeiros

UFOP - DECSI

CSI701 / CSI457

Objetivos

Ao final desta aula, você deverá ser capaz de:

- formular um problema como um problema de busca;
- distinguir agente reativo de agente solucionador de problemas;
- compreender busca cega e busca heurística;
- comparar BFS, DFS, busca de custo uniforme, gulosa e A^* ;
- interpretar o papel de uma heurística;
- analisar algoritmos por completude, otimalidade, tempo e espaço.

Base conceitual: AIMA, Capítulo 3

Pergunta central

Como um agente escolhe uma sequência de ações quando não sabe imediatamente como atingir o objetivo?

- Navegação em mapas;
- Solução de quebra-cabeças;
- Planejamento de rotas;
- Resolução de problemas em espaços discretos.

Resolver um problema = buscar uma sequência de ações

Agente solucionador de problemas

Em ambientes:

- determinísticos,
- totalmente observáveis,
- estáticos,
- discretos,
- conhecidos,

um agente pode deliberar antes de agir.

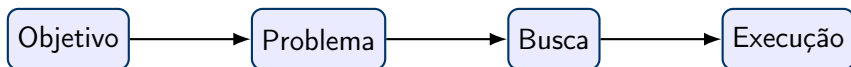
objetivo $\rightarrow$ formulação do problema $\rightarrow$ busca $\rightarrow$ execução

Mensagem central

Aqui, a inteligência está em modelar e explorar o espaço de possibilidades.

As quatro etapas do agente

- 1 **Formulação do Objetivo:** definir o objetivo;
- 2 **Formulação do Problema:** definir estados, ações e custos;
- 3 **Busca:** procurar uma sequência de ações;
- 4 **Execução:** executar a solução.



Formulação formal do problema

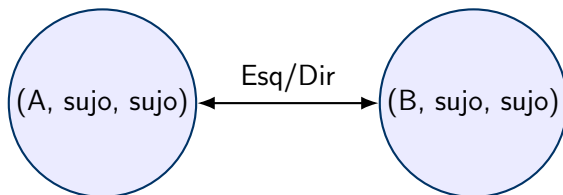
Um problema de busca pode ser descrito por:

$$\langle S, A, T, s_0, G, c \rangle$$

- S : conjunto de estados;
- A : conjunto de ações;
- T : modelo de transição;
- s_0 : estado inicial;
- G : teste de objetivo;
- c : custo de caminho.

Exemplo 1: mundo do aspirador

- estados: posição do agente + sujeira em cada sala;
- ações: Esq, Direita, Aspirar;
- objetivo: todas as salas limpas;
- custo: 1 por ação.



Discussão

Por que esse exemplo é útil para introduzir formulação de problemas?

Exemplo 2: 8-puzzle

- estado: arranjo das peças;
- ações: mover o vazio;
- objetivo: alcançar a configuração-alvo;
- custo: número de movimentos.

$$\begin{array}{ccc} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & _ & 5 \end{array} \rightarrow \begin{array}{ccc} 1 & 2 & 3 \\ 8 & _ & 4 \\ 7 & 6 & 5 \end{array}$$

Exemplo 3: mapa da Romênia

Problema clássico do AIMA (Capítulo 3)

Sair de Arad e chegar a Bucareste com custo mínimo.

- estado: cidade atual;
- ação: ir para uma cidade vizinha;
- custo: distância rodoviária;
- objetivo: chegar a Bucareste.

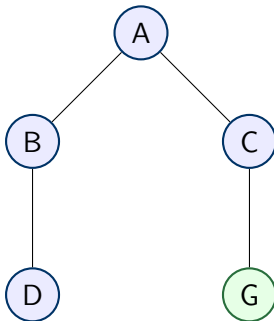
Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucareste

Árvore de busca vs. espaço de estados

Distinção

estado $\neq$ nó da árvore de busca

- o espaço de estados descreve o problema;
- a árvore de busca descreve a exploração feita pelo algoritmo.



Estrutura geral de um algoritmo de busca

- 1 inicializar a fronteira com o estado inicial;
- 2 selecionar um nó da fronteira;
- 3 testar se é objetivo;
- 4 expandir sucessores;
- 5 inserir sucessores na fronteira;
- 6 repetir até sucesso ou falha.

A estratégia de busca é a regra usada para escolher o próximo nó

Critérios de avaliação

Ao comparar algoritmos de busca, perguntamos:

- **Completa?** Encontra solução se ela existir?
- **Ótimo?** Encontra a melhor solução?
- **Tempo?** Quantos nós expande?
- **Espaço?** Quanta memória consome?

b = fator de ramificação, d = profundidade da solução

Busca Cega ou Não-Informada

Definição

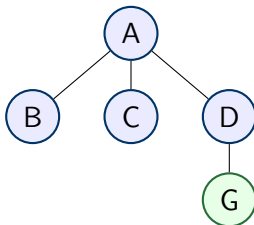
Busca cega (*uninformed search*) não usa conhecimento adicional sobre o quão perto um estado está do objetivo.

- usa apenas a definição formal do problema;
- não possui heurística;
- explora sistematicamente o espaço de busca.

Busca em Largura (Breadth-First Search)

- expande os nós por níveis;
- usa fila FIFO;
- é completa;
- é ótima se os custos forem uniformes.

$$\text{Tempo} = O(b^d), \quad \text{Espaço} = O(b^d)$$



Busca em Profundidade (Depth-First Search)

- expande o nó mais profundo primeiro;
- usa pilha LIFO;
- consome pouca memória;
- não é ótima;
- pode não terminar em espaços infinitos.

$$\text{Tempo} = O(b^m), \quad \text{Espaço} = O(bm)$$

m = profundidade máxima

Busca de Custo Uniforme

Ideia

Expandir o nó com menor custo acumulado:

$$f(n) = g(n)$$

- generaliza BFS para custos diferentes;
- é completa com custos positivos;
- é ótima.

Comparação entre estratégias cegas

Estratégia	Completa	Ótima	Tempo	Espaço
BFS	sim	sim*	$O(b^d)$	$O(b^d)$
DFS	não*	não	$O(b^m)$	$O(bm)$
Custo uniforme	sim	sim	exponencial	exponencial

* sob hipóteses apropriadas

Limitação da Busca Cega

Mensagem central

A busca cega não sabe “para onde” o objetivo parece estar.

- pode expandir muitos nós irrelevantes;
- torna-se cara em problemas maiores;
- motiva o uso de conhecimento adicional.

Busca Informada (Heurística)

Definição

Busca informada usa conhecimento adicional específico do problema para guiar a exploração.

$h(n)$ = custo estimado do nó n até o objetivo

- reduz expansão desnecessária;
- pode acelerar drasticamente a busca;
- depende da qualidade da heurística.

Best-First Search

Ideia geral

Selecionar para expansão o nó com melhor valor de uma função de avaliação:

$$f(n)$$

- a escolha de f define a estratégia;
- busca gulosa e A^* são casos particulares.

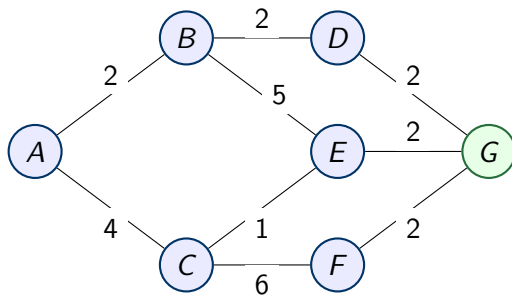
Busca Gulosa (Greedy Best-First)

$$f(n) = h(n)$$

- escolhe o nó aparentemente mais próximo do objetivo;
- frequentemente encontra solução rápido;
- não é, em geral, ótima;
- pode se perder por heurísticas ruins.

Exemplo prático: cidades e heurística

Desejamos ir de A até G.



Objetivo: encontrar o menor caminho de A até G

$$f(n) = g(n) + h(n)$$

- $g(n)$: custo real já percorrido;
- $h(n)$: estimativa do custo restante;
- combina prudência e antecipação.

Resultado clássico

Se h é admissível, A* com *tree search* é ótimo.

Admissibilidade e consistência

Admissível

$$h(n) \leq h^*(n)$$

Nunca superestima o custo ótimo restante.

Consistente

$$h(n) \leq c(n, n') + h(n')$$

Consistência $\Rightarrow$ admissibilidade

Exemplo prático: 8-puzzle e heurísticas

Duas heurísticas clássicas:

- h_1 : número de peças fora do lugar;
- h_2 : soma das distâncias de Manhattan.

$$h_2 \geq h_1 \quad (\text{em geral, mais informativa})$$

Ideia

Uma heurística melhor tende a reduzir o fator de ramificação efetivo.

Qualidade da heurística

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

onde:

- N : número de nós gerados;
- d : profundidade da solução;
- b^* : fator de ramificação efetivo.

Leitura

Quanto mais próximo de 1 for b^* , melhor é a heurística.

Comparação final

Estratégia	Informação extra	Ótima	Comentário
BFS	não	sim*	custos uniformes
DFS	não	não	pouca memória
Gulosa	sim (h)	não	rápida, mas míope
A*	sim ($g + h$)	sim	se h admissível

Atividade

Em grupos, simulem manualmente a expansão de nós para:

- BFS
- Busca Gulosa
- A*

no problema do grafo de cidades apresentado.

Roteiro da atividade

Cada grupo deve:

- 1 registrar a ordem de expansão dos nós;
- 2 calcular $g(n)$, $h(n)$ e $f(n)$ quando necessário;
- 3 identificar o caminho encontrado;
- 4 comparar custo da solução e esforço de busca;
- 5 responder: quando a heurística ajudou?

Uso crítico de IA na atividade

Permitido

Usar IA somente para:

- revisar cálculos;
- gerar novos grafos de teste;
- comparar definições conceituais.

Não suficiente

Pedir a solução pronta e apenas copiar a resposta.

IA explica → aluno testa → aluno compara → aluno justifica

Resolver problemas = buscar em um espaço de estados

Busca cega = exploração sem direção

Busca heurística = exploração guiada por conhecimento

Mensagem central

O núcleo da inteligência algorítmica está em modelar bem o problema e explorar bem o espaço de soluções.